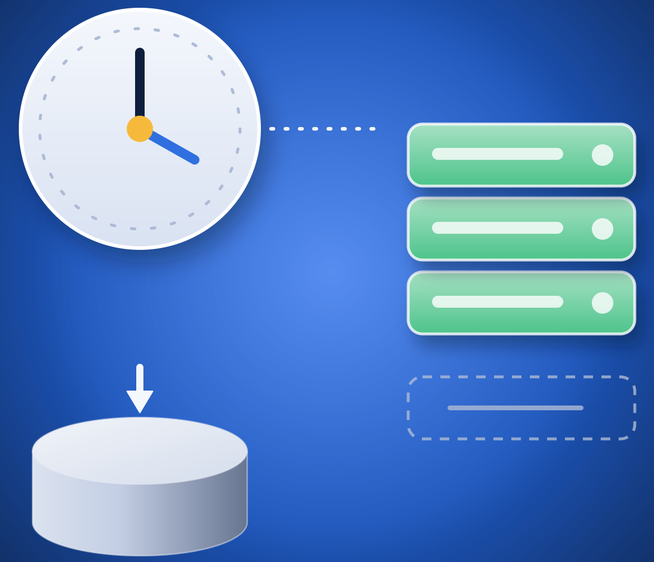
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 10

Backups, Archives and cron

Lab 10 guide: package a project, prove it can be restored, keep only three backups, and learn why cron is not your shell.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Create, list and restore a project archive, and tell archiving from compression.
2. Keep only the newest three completed archives that this lab made, and nothing else.
3. Run the script in a bare environment, and check and clean up a practice cron job when cron is available.

THIS LAB

Lab 10

First the concepts, with examples. Then the lab on the server. Scheduling the real backup, log rotation and a health snapshot are optional extensions, so they come last.

DISCUSSION

You made a backup last week. Can you restore it?

A file that exists is not a backup that works. How would you find out before you need it?

CONCEPTS FIRST

Archives, retention and cron

Four ideas: tar joins files, restoring proves a backup, retention must stay narrow, and cron runs in a bare environment.

tar joins files, gzip shrinks them

student@server: ~/oslab-work/lab10

```
$ tar -cf demo.tar -C "$PWD" project
$ tar -tf demo.tar
$ tar -czf demo.tar.gz -C "$PWD" project
$ tar -tzf demo.tar.gz
```

tar combines files and folders into one archive. gzip compresses the bytes of that archive.

-c creates, -t lists, -x extracts, -f names the archive file, and -z adds gzip.

-C "$PWD" project stores the folder as project/..., so the paths inside do not depend on where you stood.

A very small archive can be larger when compressed, so no compression ratio is promised.

A backup is proven by restoring it

student@server: ~/oslab-work/lab10

```
$ mkdir -p restore-demo
$ tar -xzf demo.tar.gz -C restore-demo
$ cmp -- project/report.txt restore-demo/project/report.txt
$ cmp -- project/config.ini restore-demo/project/config.ini
```

Extract into a new, empty folder you own. Then compare each file with the original.

cmp prints nothing and returns 0 when the bytes are equal. Creating an archive without error does not show it can be restored.

CAREFUL

Only your own archives

Extract only archives you made. Never extract an unknown archive on the server.

Retention: keep three, touch nothing else

RULE

The policy

Keep the newest three completed archives that the script made. Remove older ones.

SCOPE

Narrow on purpose

Look only in your own backups/ folder. Consider only names like backup-*.tar.gz. Leave every other file alone.

HOW

A safe deletion

Sort the names, skip the first three, and remove the rest. Quote every path. Never write rm $(ls ...).

What cron gives a script

Field	Range	Example
Minute	0 to 59	*/5 is every five minutes
Hour	0 to 23	2 is 02:00
Day of month	1 to 31	*
Month	1 to 12	*
Day of week	0 to 7	1 is Monday

A cron line has five time fields and a command. There is no year, so a date entry comes back every year unless something guards or removes it.

Cron starts the job in your home folder, with a small environment. Your shell's PATH and settings are not there.

Use absolute paths, and let the script find its own folder.

LAB 10

Backups you can restore

QuantumTech needs repeatable backups that work. Package a project, keep three, and test under a bare environment.

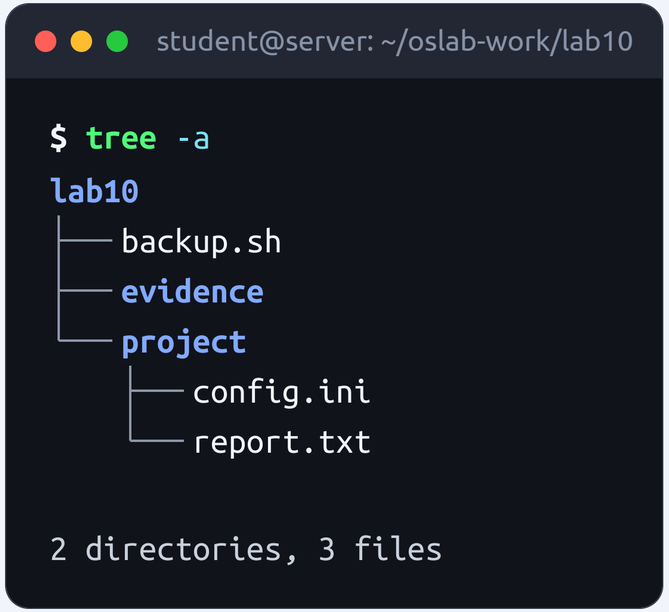
The scenario

SCENARIO

QuantumTech's backups

Package a small project, keep only three completed archives that this lab made, and diagnose how the script runs in a restricted scheduling environment.

Dated, graded cron jobs from the old lab are replaced by jobs you can watch in class.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Levels 0 and 1, guided: archive and restore
25 to 35	Written prediction
35 to 60	Level 2: the backup script and retention

Minutes	What happens
60 to 70	Levels 3 and 5: cron and the environment
70 to 85	Tests and feedback
85 to 100	Individual changed-case checkpoint
100 to 120	Correction, cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Do not enable services or edit /etc/crontab. Do not extract unknown archives.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

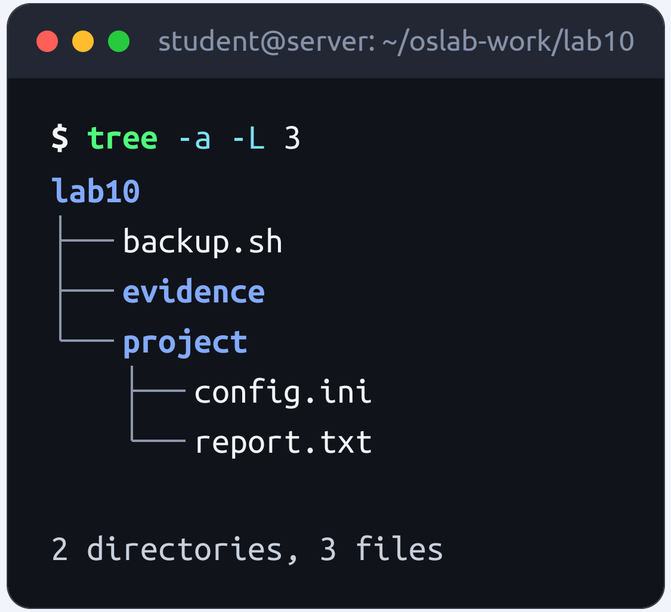
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab10
$ cd "$OSLAB_WORKSPACE/lab10"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab10 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



backup.sh is an editable skeleton.

project/ is the small project to back up: report.txt and config.ini.

The helper also writes a hidden file, .oslab-managed.json. Leave it as it is.

GUIDED

Archive and restore

Levels 0 and 1, with the instructor: fifteen minutes.

Step 1: look at the project

student@server: ~/oslab-work/lab10

```
$ find project -maxdepth 2 -type f
$ cat project/report.txt project/config.ini
$ mkdir -p backups
```

find lists the files of the project, down to two levels. cat shows what they hold.

mkdir -p backups makes the folder that will hold your archives.

Step 2: an archive, then a compressed one

student@server: ~/oslab-work/lab10

```
$ tar -cf demo.tar -C "$PWD" project
$ tar -tf demo.tar
$ tar -czf demo.tar.gz -C "$PWD" project
$ tar -tzf demo.tar.gz
$ ls -l demo.tar demo.tar.gz
```

Both archives use the same controlled member tree: project/....

TRY

Observe

Which file is smaller? Is that always true for very small archives? What does -C make independent of your current folder?

Step 3: restore the demo and compare

student@server: ~/oslab-work/lab10

```
$ mkdir -p restore-demo
$ tar -xzf demo.tar.gz -C restore-demo
$ cmp -- project/report.txt restore-demo/project/report.txt
$ cmp -- project/config.ini restore-demo/project/config.ini
```

No output and status 0 mean the contents match.

THINK

Observe

Is a successful archive creation the same as checking that you can recover? What did you check just now?

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

A cron entry runs backup.sh with no path, and the script assumes its working folder is the project. Must it work because it ran in your interactive shell? Name two assumptions you would verify.

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run the command. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

LEVEL 2

The backup script and retention

Twenty-five minutes, from minute 35 to 60. Work alone.

Step 1: the starting script

backup.sh

```
#!/usr/bin/env bash
set -euo pipefail
work=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
mkdir -p "$work/backups"
stamp=$(date -u +%Y%m%dT%H%M%S%N)
final="$work/backups/backup-$stamp-$$.tar.gz"
pending=$(mktemp "$work/backups/.pending-XXXXXX")
trap 'rm -f -- "$pending"' EXIT
tar -czf "$pending" -C "$work" project
tar -tzf "$pending" >/dev/null
mv -- "$pending" "$final"
# TODO: retain only the newest three completed backup-*.tar.gz files.
printf 'created %s\n' "$final"
```

Save it as backup.sh and run it with bash backup.sh. set -euo pipefail stops the script on an error, on an unset variable, or when a command in a pipeline fails.

work is the folder that holds the script (BASH_SOURCE[0] is its path, dirname drops the file name, pwd -P prints the real folder), so it works from any folder.

The archive is built under a temporary name and moved into place only when complete. trap removes the temporary file if something fails. The orange TODO is your work: retention.

Step 2: restore the new archive, and save the proof

student@server: ~/oslab-work/lab10

```
$ archive=$(find backups -maxdepth 1 -type f -name 'backup-*.tar.gz' | sort | tail -n 1)
$ mkdir -p restore-backup
$ tar -tzf "$archive" | tee evidence/restore.txt
$ tar -xzf "$archive" -C restore-backup
$ cmp -- project/report.txt restore-backup/project/report.txt
$ printf 'report comparison=%s\n' "$?" | tee -a evidence/restore.txt
```

The first line finds the newest completed archive: sort puts the names in order, and tail -n 1 takes the last.

List it first, then extract and compare. Do the same for config.ini. tee saves a copy as evidence.

Step 3: write the retention

Requirements

- Look only in your own backups/ folder.
- Consider only the backup-*.tar.gz names this script makes.
- Keep the newest three. Leave all other names alone.
- Do not use an unquoted rm $(ls ...).

YOUR TURN

You write it

Hints: define the owned names before you delete. Put the completed archives in a sorted array. Loop only over the entries after the first three, with every path quoted.

Step 4: four backups, and a file that must survive

student@server: ~/oslab-work/lab10

```
$ printf 'unrelated; preserve me\n' > backups/keep-me.txt
$ bash backup.sh
$ printf 'report v2\n' > project/report.txt
$ bash backup.sh
$ bash backup.sh
$ find backups -maxdepth 1 -type f
```

Count the first run and these three as four backups. Three archives and keep-me.txt must remain.

Change the project once between backups, so that the versions differ. Never delete by a broad path, and never scan your whole home.

LEVELS 3 AND 5

Cron and the environment trap

Ten minutes, from minute 60 to 70.

Step 1: run it under a bare environment

student@server: ~/oslab-work/lab10

```
$ cd "$HOME"
$ env -i HOME="$HOME" PATH=/usr/bin:/bin /bin/bash "$OSLAB_WORKSPACE/lab10/backup.sh"
$ cd "$OSLAB_WORKSPACE/lab10"
```

env -i starts the command with an empty environment, and then only HOME and PATH are set. This is close to what cron gives a job.

This tests path and environment assumptions. It is not real cron delivery.

The script's own-folder line should still find the project.

Step 2: a practice cron job

student@server: ~/oslab-work/lab10

```
$ lab10-cron status
$ lab10-cron install
$ crontab -l
```

The helper adds one uniquely marked entry of your own, with an absolute path, and leaves the others alone.

It writes a timestamp every minute. It does not run your backup.

CAREFUL

Wait up to two minutes

cron.log should show a timestamp. Cron may be missing in WSL. Then say that scheduling was not available, and use the bare-environment test.

Four cases, and what is untested

Fifteen minutes, from minute 70 to 85.

The four cases

Case	Expected evidence
Archive and restore	The member list, and restored files with the same bytes
Four backups in a row	Three completed archives, and the unrelated file untouched
Another folder, bare environment	The backup still finds its source, and retention still holds
Personal cron, if available	A timestamp in the log, and only the marked entry removed afterwards

Save the retention, bare-environment and cron or fallback evidence in evidence/automation.txt. Write what you did not test: a source that changes during the backup, hostile archives, system-wide scheduling, time zones.

Save the evidence

student@server: ~/oslab-work/lab10

```
$ {
$   find backups -maxdepth 1 -type f
$   printf 'completed archive count='
$   find backups -maxdepth 1 -type f -name 'backup-*.tar.gz' | wc -l
$   cat backups/keep-me.txt
$ } > evidence/automation.txt
```

Then add the bare-environment result and the cron observation, or your fallback statement, with an editor.

If cron ran, add lab10-cron status before and after the removal. The saved crontab should hold only the practice line.

An empty log in the first seconds is not a failure. Check cron is available, the paths are absolute, and stderr.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

A cron entry finds /bin/bash, but runs a script that contains tar -czf backup.tar.gz project without changing folder. Explain the likely path fault, and one robust correction. Why does a five-field date entry not mean 'run once'?

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

The instructor collects your answer before discussing the public key.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) How did you show that the backup can be restored? (b) What stopped retention from deleting unrelated data? (c) Did you see real scheduling, or only the fallback environment?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab10/evidence"
$ cd "$OSLAB_WORKSPACE/lab10"
$ cp -- backup.sh "$SUBMISSION_REPO/lab10/"
$ cp -- evidence/restore.txt evidence/automation.txt "$SUBMISSION_REPO/lab10/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Remove your practice cron entry before you finish. automation.txt holds the retention, environment and cron or fallback evidence.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab10
├── backup.sh
├── evidence
│   ├── automation.txt
│   └── restore.txt
└── README.md

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
A restorable archive, and keep-three retention in the right scope	3
Restore, retention, unrelated-file and bare-environment evidence	2
Archive versus compression, paths and scheduling limits explained; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
rm $(ls backups)	Unrelated files removed	Sort the matching names, quote every path
Relative paths in the script	It works in your shell, fails in cron	Find the script's own folder first
Trusting a backup you never restored	A corrupt or incomplete archive	Extract it and compare with cmp
Leaving the practice cron entry	A job that keeps running after the lab	lab10-cron remove, then check crontab -l

BEYOND THE CORE

Schedule it, rotate logs, a health snapshot

Optional extensions. They need your own cron and dates the instructor gives you.

Lab 10

Schedule the repaired backup

student@server: ~/oslab-work/lab10

```
$ crontab -l
$ */5 * * * * /bin/bash /home/student/oslab-work/lab10/backup.sh >> /home/student/oslab-work/lab10/cron.log 2>&1
```

Install it only after backup.sh works under the bare environment. Use absolute paths for the shell, the script and the log.

2>&1 sends error messages to the log too, so you can see why a run failed. Remove the entry when you have seen it run.

Logs, a health snapshot, a guarded date job

LOGS

Rotate your own logs

Move an owned log to a dated name and start a new one, but never while a job is writing it.

HEALTH

A health snapshot

A small script that saves df -h, uptime and the newest backup name to one file, and is safe to run again.

DATES

A guarded date job

Cron has no year. Use dates and the Asia/Phnom_Penh time zone from the instructor, and make the script check the date itself.

Check which time zone the cron daemon on the server uses. Changing TZ in your shell does not change how the schedule is read.

Key takeaways

01

Archive, then compress

tar joins the files. gzip shrinks the bytes. -C keeps the paths inside the archive stable.

02

Restore to prove it

A backup is a backup when you have extracted it and compared the bytes.

03

Keep the scope narrow

Retention touches only the names this script made, in your own folder.

04

cron is a bare shell

Absolute paths, no assumptions about your folder, and test with env -i first.

Next session

COMING UP

Lab 11 (bonus): disk management utilities

Look at disks, partitions and filesystems, with tools that only read.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 14, File-System Implementation.